

Security Insider Lab I

Lab 2: Machine Learning for Network Attack Detection

Group 8

Justlyn Kaniyanthara Johnson	121024
Aayush Kapoor	120957

---

6. November 2026

Contents

1 Introduction **1**

2 Task 1: Network Flows with CICFlowMeter **1**
2.1 What a Flow Is . . . 1
2.2 Capture Setup . . . 1
2.3 The IPv6 Problem . . . 1
2.4 The Final Capture . . . 3

3 Task 2: The CIC-IDS2017 Dataset **4**

4 Task 3: Data Pre-processing **4**
4.1 Results . . . 5
4.2 Interpretation . . . 7

5 Task 4: Machine Learning Classifiers **9**
5.1 Setup . . . 9
5.2 Binary Results . . . 9
5.3 Multiclass Results . . . 10
5.4 Interpretation . . . 10

6 Task 4: Real-Time Classification **11**
6.1 Setup . . . 11
6.2 Correctness: Parity Test . . . 12
6.3 Results on Our Capture . . . 12
6.4 The False Positives . . . 13
6.5 Latency and Scope . . . 14

7 Discussion and Conclusion **14**

Bibliography **16**

Eidesstattliche Erklärung **17**

Abstract

In the second lab of the Security Insider Lab, we detect network attacks from flow features with machine learning. We first recorded our own traffic and converted it into flows with CICFlowMeter. Our first captures were mostly IPv6 and produced bogus flows, because CICFlowMeter force-parses non-IPv4 frames as IPv4. With IPv6 disabled, we obtained 569 flows, 547 of them valid TCP/UDP flows. As training data we chose the Thursday web-attack file of CIC-IDS2017 (Brute Force, XSS, SQL injection) instead of the suggested DDoS file, because web attacks look like normal HTTP and are harder to detect. Pre-processing removed infinite values, duplicate rows and constant and identical columns, which left 164,179 flows with 60 features. We then made a stratified split, min-max scaled the features and selected the 20 best with χ^2. We trained Logistic Regression and Decision Tree classifiers for a binary and a four-class target. The default Decision Tree is best: it detects all 643 attacks of the test split with 18 false positives (attack F1 0.986) and reaches a multiclass macro-F1 of 0.729, whereas default multiclass Logistic Regression reaches 98.5 % accuracy while detecting no attack. Applied flow by flow to our own benign capture, the Decision Tree raised 3 false alarms in 547 flows (0.55 %, about 15 times the test-split rate), caused by shortcut features such as the destination port and the initial TCP window size.

List of Figures

2.1 CICFlowMeter in offline mode converting the capture, under its original file name `benign_lab2_3.pcap`, with the default timeouts (120 s flow, 5 s activity): 81,110 packets read (81,109 valid, 1 discarded), the same packet count as `task1_lab2.pcapng`. The reported "Total 570 flows" counts the lines of the output CSV including its header, i.e. 569 flows. 2
2.2 Disabling IPv6 on the capturing laptop before the third capture; `sysctl` confirms `net.ipv6.conf.all.disable_ipv6 = 1`. 2
2.3 Protocol hierarchy of `task1_lab2.pcapng` in Wireshark: 99.94 % of the frames are IPv4, only 24 are IPv6. 3
2.4 First rows of `task2_lab2.pcap_Flow.csv` in LibreOffice Calc: flow ID, endpoints, protocol, timestamp and the first features. (The `Label` column further to the right reads `NeedManualLabel`.) 3

4.1 Cleaning steps as logged by the pre-processing script (cf. Table 4.1). 6
4.2 Label analysis as printed by the pre-processing script (cf. Tables 3.1 and 4.3). 7
4.3 The 20 best features by χ^2 as printed by the pre-processing script (cf. Table 4.5). . . . 7
4.4 Top-20 flow features by χ^2 score for binary classification (BENIGN vs. ATTACK), computed on the min-max-scaled training set. 8

5.1 Console output of the training script for the chosen model, the default Decision Tree (binary). 10
5.2 Confusion matrices of four of the eight models (test split). 11

6.1 Real-time architecture. Flows are streamed row by row in timestamp order. 12
6.2 Test evidence for Tasks 3 and 4. 12

6.3 Output of the real-time detector: streamed flow verdicts, the summary (569 read, 22 dropped, 547 classified, 3 alerts = 0.55 %) and the three alerts in red. 13
6.4 The seven parallel Steam requests in Wireshark (display filter at the top): `GET /appinfo/…` to the Steam CDN at 92.226.1.27, all answered with `200 OK`. The host name, `clientconfig.akamai.steamstatic.com`, comes from the HTTP Host header. 14

List of Tables

2.1 The three captures. Only the third, with IPv6 disabled, yields mostly valid flows. . . . 2
2.2 How non-IPv4 frames turn into fake IP addresses in the CICFlowMeter output. 3
2.3 Flows of `task2_lab2.pcap_Flow.csv`. The 22 flows with protocol 0 are not TCP/UDP flows and are dropped before classification. 4

3.1 Label distribution of the Thursday file, raw and after the cleaning steps of Chapter 4. 4

4.1 Cleaning steps. The column counts include the `Label` column, so the 61 remaining columns are 60 features plus `Label`. Loading and label fixing happen before step 1. . . 6
4.2 Removed columns. For identical pairs, the first column is kept and the second one dropped. 6
4.3 Stratified 70/30 split (`random_state=42`). 7
4.4 Six example features of the training set before and after min-max scaling. 7
4.5 The 20 best features by χ^2 score (min-max-scaled training set), for the binary and the multiclass target. 8

5.1 Binary results on the test split (Prec., Rec., F1 refer to the attack class; FP/FN are false positives and false negatives). Best model in bold. 9
5.2 Multiclass results on the test split. The columns BENIGN to SQLi are per-class F1 scores. 10

6.1 Real-time run on `task2_lab2.pcap_Flow.csv`. 13

Listings

4.1 Core cleaning steps of `clean()` in `src/lab2ids/data.py` (steps 2–6; logging calls omitted) 5

5.1 `build_pipeline()` and `make_classifier()` in `src/lab2ids/models.py` 9

6.1 Excerpt of `Detector.process()` in `src/lab2ids/realtime/detector.py`: column mapping, Protocol-0 drop, feature check, inf/NaN replacement and prediction (some lines omitted) 12

1 Introduction

In the second lab of the Security Insider Lab, we detect network attacks with machine learning. Instead of inspecting packets one by one, we describe each network connection by a vector of statistical features (a *flow*) and train classifiers to tell benign flows from attack flows. The lab has four tasks:

1. **Task 1:** we capture our own traffic and convert it into flows with CICFlowMeter.
2. **Task 2:** we choose and analyze a labeled flow dataset, CIC-IDS2017.
3. **Task 3:** we pre-process the dataset: we handle infinite and missing values, scale the features, and select the 20 most informative ones.
4. **Task 4:** we train Logistic Regression and Decision Tree classifiers, evaluate them, and apply the best model to the flows of our own capture.

Dataset choice. The lab slides suggest the Friday DDoS file of CIC-IDS2017. We deliberately chose a different one, `Thursday-WorkingHours-Morning-WebAttacks`. Web attacks (Brute Force, cross-site scripting and SQL injection against a web application) are low-volume, application-layer attacks. Their flows look like ordinary HTTP requests. For a volumetric DDoS attack, the traffic volume alone reveals the attack. Web attacks are therefore harder, and in our view more interesting, to detect from flow features.

Structure. Chapter 2 describes how we produced flows from our own capture and the problems we ran into. Chapter 3 introduces the dataset, Chapter 4 the pre-processing, and Chapter 5 the classifiers and their evaluation. Chapter 6 applies the best model to our own flows, and Chapter 7 discusses the results and limitations. All code is in our `lab2ids` Python package; the listings in this report are excerpts of the real source files.

2 Task 1: Network Flows with CICFlowMeter

2.1 What a Flow Is

A flow summarizes a bidirectional exchange of packets between two endpoints, identified by source and destination IP address, ports and protocol. CICFlowMeter [Las+17; CFM] reads a packet capture and writes one row per flow with 84 columns: identifying metadata (flow ID, addresses, ports, protocol, timestamp), 77 numeric features (packet counts and lengths, inter-arrival times, TCP flag counts, window sizes, idle and active times) and a label column. A flow ends when a FIN or RST is seen, or when a timeout expires: 5 s without activity (activity timeout) or 120 s of total flow duration (flow timeout). CICFlowMeter only writes flows with at least two packets, so single-packet exchanges, such as a lone DNS query or a traceroute probe, do not appear in the output.

2.2 Capture Setup

We recorded our own traffic with Wireshark while browsing, resolving names and running the Steam client. The final capture, `task1_lab2.pcapng`, has 81,110 frames over 347.4 s (2026-11-04, 16:05–16:10). We converted it offline with CICFlowMeter to `task2_lab2.pcap_Flow.csv`. This file is the real-time input of Chapter 6. Since the traffic is ordinary browsing, we treat all flows as benign by construction.

2.3 The IPv6 Problem

Our first two captures produced CSV files in which most flows were garbage: protocol 0, port 0 and implausible IP addresses. Table 2.1 compares the captures. In the first two, 89–98 % of the frames were IPv6. After we disabled IPv6 on the capturing laptop for the third capture with `sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1` (Figure 2.2), only 24 IPv6 frames (0.03 %) remained and the share of valid flows rose from 15 % and 5 % to 96 %.

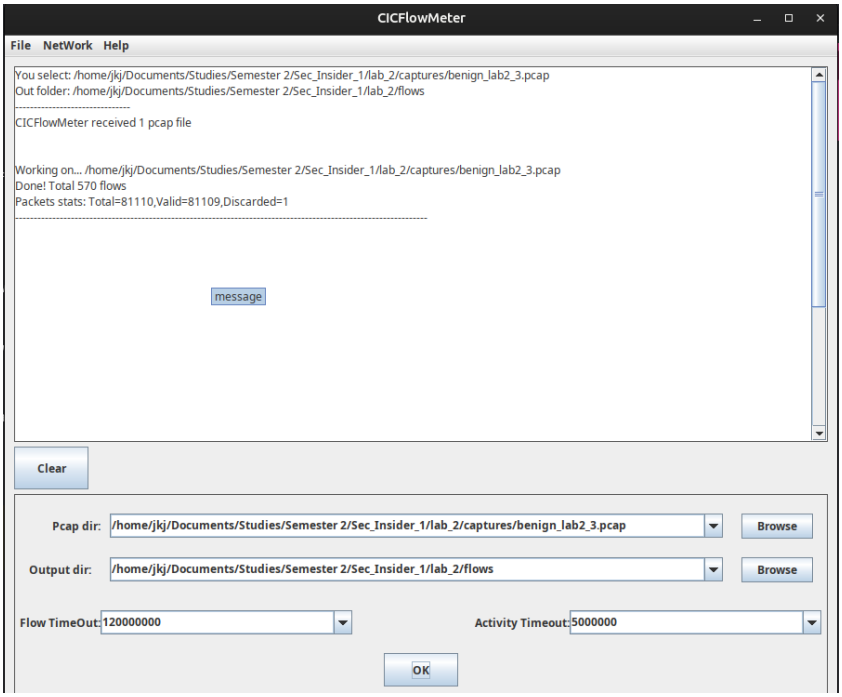


Figure 2.1: CICFlowMeter in offline mode converting the capture, under its original file name `benign_lab2_3.pcap`, with the default timeouts (120 s flow, 5 s activity): 81,110 packets read (81,109 valid, 1 discarded), the same packet count as `task1_lab2.pcapng`. The reported "Total 570 flows" counts the lines of the output CSV including its header, i.e. 569 flows.

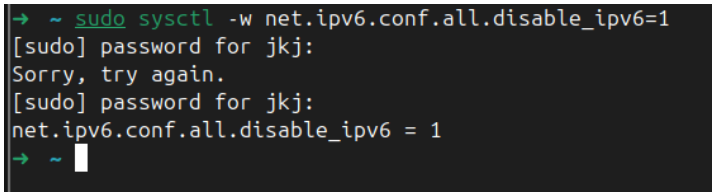


Figure 2.2: Disabling IPv6 on the capturing laptop before the third capture; `sysctl` confirms `net.ipv6.conf.all.disable_ipv6 = 1`.

Capture	Frames	IPv6 frames	Flows	Protocol 0	Valid TCP/UDP
1st (since deleted)	129,548	126,904 (98 %)	532	454	78
2nd (since deleted)	98,780	88,458 (89.5 %)	1,327	1,260	67
3rd, IPv6 off (final)	81,110	24 (0.03 %)	569	22	547

Table 2.1: The three captures. Only the third, with IPv6 disabled, yields mostly valid flows.

Root cause. We traced the problem in the CICFlowMeter source. IPv6 support is switched off in the code: `readIP6 = false` is hard-coded in `Cmd.java` (line 126) and `ReadPcapFileWorker.java` (line 152). Every frame that is not IPv4 (IPv6, ARP, HomePlug powerline frames) falls through to `getVPNInfo()`, which force-parses the raw frame as an IPv4 header starting at byte 0. The fake "source IP" is therefore just the EtherType plus two further bytes, and the fake "destination IP" is taken from the IPv6 flow label and payload length (Table 2.2). The IPv4 flows themselves are parsed correctly. We decided against enabling CICFlowMeter's IPv6 path, because its UDP branch wrongly reads the header length from the TCP header (`PacketReader.java`, line 204). With IPv6 off, our flows are IPv4-only, like the training data.

Fake “IP”	Bytes	Actually
`134.221.96.0`	`86 DD 60 00`	EtherType 0x86DD (IPv6) and first IPv6 header bytes
`0.0.4.214` (dst)	`00 00 04 D6`	IPv6 flow label and payload length (1238)
`136.225.0.0`	`88 E1 …`	EtherType 0x88E1 (HomePlug AV)
`8.6.x.x`	`08 06 …`	EtherType 0x0806 (ARP)

Table 2.2: How non-IPv4 frames turn into fake IP addresses in the CICFlowMeter output.

2.4 The Final Capture

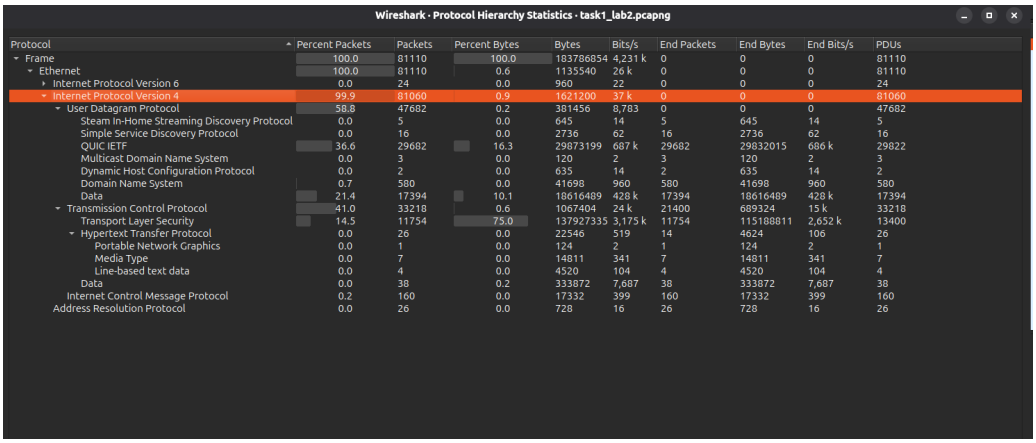


Figure 2.3: Protocol hierarchy of `task1_lab2.pcapng` in Wireshark: 99.94 % of the frames are IPv4, only 24 are IPv6.

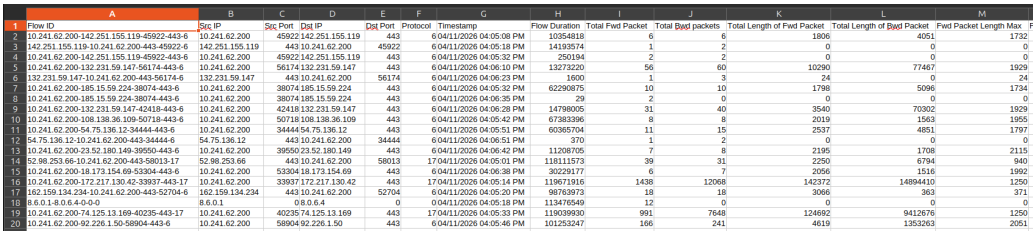


Figure 2.4: First rows of `task2_lab2.pcap_Flow.csv` in LibreOffice Calc: flow ID, endpoints, protocol, timestamp and the first features. (The `Label` column further to the right reads `NeedManualLabel`.)

The final capture contains 47,682 UDP frames (29,682 QUIC, 580 DNS), 33,218 TCP frames (11,994 TLS, 26 HTTP), 160 ICMP frames and 26 ARP frames (Figure 2.3). Its offline conversion (Figure 2.1) yields 569 flows (Table 2.3, Figure 2.4). More than half of the classified flows (289) are DNS queries to the local resolver, since every query is its own flow; 213 go to port 443 and 15 to port 80. The classified flows contain 80,705 packets, which is 99.8 % of the 80,900 IPv4 TCP/UDP frames. The missing packets are single-packet UDP exchanges, mostly traceroute probes, which CICFlowMeter does not write.

The 22 protocol-0 flows consist of 15 real ICMP flows (CICFlowMeter only sets the protocol number for TCP and UDP), 4 flows from left-over IPv6 frames and 3 from ARP frames, the last two groups mis-parsed as described above. The real-time detector drops all 22 (3.9 %) and logs the count. There are no infinite or NaN values in the remaining flows, and all 77 features map to the training feature names (Section 6.1). About 70 % of the valid flows are UDP (DNS, QUIC and Steam). The application mix (e.g. QUIC and Steam) and the network itself differ from the 2017 testbed, so we expect domain shift (Chapter 6).

Quantity	Value
Total flows	569
TCP flows (6)	162
UDP flows (17)	385
Protocol 0 flows (dropped)	22
Flows classified (TCP + UDP)	547
Packets in classified flows	80 705
Top-1 destination port	53 (289 flows)
Top-2 destination port	443 (213 flows)
Top-3 destination port	80 (15 flows)

Table 2.3: Flows of `task2_lab2.pcap_Flow.csv`. The 22 flows with protocol 0 are not TCP/UDP flows and are dropped before classification.

3 Task 2: The CIC-IDS2017 Dataset

CIC-IDS2017 was created by the Canadian Institute for Cybersecurity at the University of New Brunswick [SLG18; CIC]. The flows were extracted with CICFlowMeter [Las+17]. We use the pre-computed "MachineLearningCSV" file `Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv` (52 MB). It has 170,366 flows and 79 columns: 78 numeric flow features and the `Label`. It contains no flow ID, IP address or timestamp columns, but `Destination Port` is present. According to the dataset paper, the attacks (Brute Force, cross-site scripting and SQL injection) were run against the Damn Vulnerable Web Application (DVWA) and automated with Selenium on Thursday morning.
The dataset is extremely imbalanced (Table 3.1): attacks are only 1.28 % of the raw flows, and there are just 21 SQL injection flows. A classifier that always answers "BENIGN" would already be 98.7 % accurate. We therefore report per-class precision, recall and F1, the macro-F1 and confusion matrices throughout, never accuracy alone. All 2,180 attack flows target destination port 80, while benign traffic is spread over many ports (53, 443, 80, 123, 22, …). Models can exploit this as a shortcut (Section 4.2 and Chapter 7).

Label	Raw	Share	Cleaned	Share
BENIGN	168 186	98.72 %	162 036	98.69 %
Web Attack – Brute Force	1 507	0.88 %	1 470	0.90 %
Web Attack – XSS	652	0.38 %	652	0.40 %
Web Attack – SQL Injection	21	0.01 %	21	0.01 %
Total	**170 366**	**100.00 %**	**164 179**	**100.00 %**

Table 3.1: Label distribution of the Thursday file, raw and after the cleaning steps of Chapter 4.

4 Task 3: Data Pre-processing

The task asks us to analyze the labels, handle infinite and NaN values, scale the features with min-max scaling and select the 20 best features with `SelectKBest`. We implemented this in Python 3.12 with pandas and scikit-learn [Ped+11] (modules `data.py` and `features.py`, script `task3_preprocess.py`). The steps are, in this order:

1. **Load and analyze labels.** 65 of the 79 column names carry stray leading spaces and are stripped. The en dash in the attack labels was stored as an invalid byte and decodes as U+FFFD ("`Web Attack` `&#xFFFD;` `XSS`"). We read the file as UTF-8 with replacement, map that character to "-" and collapse whitespace. Every label is validated against the four expected classes.
2. **Drop the duplicate column.** `Fwd Header Length` appears twice in the header, a known artifact of the CICFlowMeter export. We drop the second copy.

3. **Infinity and NaN.** We replace $\pm\infty$ by NaN and *drop* the affected rows. Only 135 rows (0.08 %) are affected, all of them BENIGN, so dropping loses no attack information and introduces no artificial values. The infinite values occur only in `Flow Bytes/s` and `Flow Packets/s`: CICFlowMeter divides by the flow duration, which is 0 for single-packet flows.
4. **Duplicate rows.** We remove exact duplicates before splitting. Otherwise identical rows land in both train and test sets and inflate the test scores.
5. **Constant columns.** They carry no information and trigger warnings in `SelectKBest`.
6. **Identical columns.** Columns equal to an earlier one (tolerance 10^{-6}) are removed, so that the 20 selected features are distinct. The tolerance is needed for one pair, `Bwd Packet Length Mean` and `Avg Bwd Segment Size`: they are the same quantity but differ in 2 rows by 10^{-6} because of CSV rounding.
7. **Train/test split.** A stratified 70/30 split on the four-class label with `random_state=42`, done *before* anything is fitted. Both targets, binary (BENIGN vs. ATTACK) and multiclass, use the same split.
8. **Min-max scaling.** `MinMaxScaler(clip=True)` maps each feature to $[0, 1]$. It is fitted on the training set only, so no test statistics leak into training, and `clip=True` keeps unseen values, such as live traffic later, inside $[0, 1]$.
9. **Feature selection.** `SelectKBest(chi2, k=20)` on the scaled training set, once per target. We scale first because χ^2 requires non-negative input, and the raw data contains negative values: `Init_Win_bytes_*` is -1 when no window was observed, and a few durations are negative.

The column-level steps 5 and 6 use feature values only, never labels, so they leak no label information. Listing 4.1 shows the core of steps 2–6 in the code.

Listing 4.1: Core cleaning steps of `clean()` in `src/lab2ids/data.py` (steps 2–6; logging calls omitted)

```
    # 2. inf -> NaN
    feats = [c for c in df.columns if c != LABEL_COL]
    num = df[feats]
    df[feats] = num.replace([np.inf, -np.inf], np.nan)
    # 3. drop NaN rows
    before = df.shape
    mask = df[feats].isna().any(axis=1)
    bd = _breakdown(df.loc[mask, LABEL_COL])
    df = df.loc[~mask]
    # 4. duplicates
    before = df.shape
    dup = df.duplicated()
    bd = _breakdown(df.loc[dup, LABEL_COL])
    df = df.loc[~dup]
    # 5. constant columns
    before = df.shape
    feats = [c for c in df.columns if c != LABEL_COL]
    const = [c for c in feats if df[c].nunique(dropna=False) <= 1]
    df = df.drop(columns=const)
    # 6. identical columns
    before = df.shape
    feats = [c for c in df.columns if c != LABEL_COL]
    pairs = _identical_columns(df[feats])
    df = df.drop(columns=[b for _, b in pairs])
```

4.1 Results

Table 4.1 lists the cleaning steps with their effect, and Table 4.2 the removed columns. The raw file has 6,066 duplicate rows, but 14 of them contain infinite or NaN values and are already gone after step 3, so step 4 drops 6,052. The six bulk-transfer features are always 0 in this CICFlowMeter version. `Fwd PSH Flags` is identical to `SYN Flag Count`, although PSH and SYN are different TCP flags. This points

to a feature-extraction problem in the CICFlowMeter version used for CIC-IDS2017, in line with the literature [ERJ21].

#	Step	Rows	Cols	Detail
1	Drop duplicate header column	170 366→170 366	79→78	−1 column (`Fwd Header Length.1`)
2	Replace ±∞ by NaN	170 366→170 366	78→78	inf: `Flow Bytes/s` 115, `Flow Packets/s` 135; NaN: `Flow Bytes/s` 20
3	Drop NaN rows	170 366→170 231	78→78	−135 rows (all BENIGN)
4	Drop duplicate rows	170 231→164 179	78→78	−6 052 rows (6 015 BENIGN, 37 Brute Force)
5	Drop constant columns	164 179→164 179	78→68	−10 columns
6	Drop identical columns	164 179→164 179	68→61	−7 columns
	Result	**164 179**		**164 179 rows × 60 features**

Table 4.1: Cleaning steps. The column counts include the `Label` column, so the 61 remaining columns are 60 features plus `Label`. Loading and label fixing happen before step 1.



Figure 4.1: Cleaning steps as logged by the pre-processing script (cf. Table 4.1).

Kind	Columns
Constant (10)	`Bwd PSH Flags`, `Fwd URG Flags`, `Bwd URG Flags`, `CWE Flag Count`, `Fwd Avg Bytes/Bulk`, `Fwd Avg Packets/Bulk`, `Fwd Avg Bulk Rate`, `Bwd Avg Bytes/Bulk`, `Bwd Avg Packets/Bulk`, `Bwd Avg Bulk Rate`
Identical	`Fwd PSH Flags` → `SYN Flag Count` (dropped)
Identical	`RST Flag Count` → `ECE Flag Count` (dropped)
Identical	`Fwd Packet Length Mean` → `Avg Fwd Segment Size` (dropped)
Identical	`Bwd Packet Length Mean` → `Avg Bwd Segment Size` (dropped)
Identical	`Total Fwd Packets` → `Subflow Fwd Packets` (dropped)
Identical	`Total Length of Fwd Packets` → `Subflow Fwd Bytes` (dropped)
Identical	`Total Backward Packets` → `Subflow Bwd Packets` (dropped)

Table 4.2: Removed columns. For identical pairs, the first column is kept and the second one dropped.

After cleaning, 164,179 flows with 60 features remain, of which 2,143 (1.31 %) are attacks (Table 3.1), a ratio of roughly 76 benign flows per attack. Table 4.3 shows the split: the test set contains 643 attack flows, but only 6 SQL injection flows.

Table 4.4 shows six features before and after scaling (training set). Across all 60 training columns, the scaled minimum is 0 and the maximum 1 (up to floating-point rounding). The heavy-tailed durations, with maxima near the 120 s timeout, end up with small means after scaling.

The binary and multiclass feature lists share 18 of 20 features (Table 4.5; binary scores in Figure 4.4): only `Fwd IAT Total` and `Flow Duration` are specific to the binary target, and only `Average Packet Size` and `Bwd Packet Length Std` to the multiclass target. The top three features score above 1,600 each and the rest below 250, so ranks 6–20 look alike on the linear axis.

Label	Train (70 %)	Test (30 %)
BENIGN	113 425	48 611
Web Attack – Brute Force	1 029	441
Web Attack – XSS	456	196
Web Attack – SQL Injection	15	6
Total	**114 925**	**49 254**

Table 4.3: Stratified 70/30 split (`random_state=42`).

```
======================================================================
Raw label summary
======================================================================
                          count  percent  ratio_to_benign
Label
BENIGN                   168186  98.7204         1.000000
Web Attack - Brute Force   1507   0.8846         0.008960
Web Attack - XSS            652   0.3827         0.003877
Web Attack - Sql Injection   21   0.0123         0.000125
```

(a) Raw labels

```
======================================================================
Final label summary
======================================================================
                          count  percent  ratio_to_benign
Label
BENIGN                   162036  98.6947         1.000000
Web Attack - Brute Force   1470   0.8954         0.009072
Web Attack - XSS            652   0.3971         0.004024
Web Attack - Sql Injection   21   0.0128         0.000130

======================================================================
Split class counts (70/30, stratified on Label)
======================================================================
                          train   test
Label
BENIGN                   113425  48611
Web Attack - Brute Force   1029    441
Web Attack - XSS            456    196
Web Attack - Sql Injection   15      6
```

(b) Labels after cleaning and the 70/30 split

Figure 4.2: Label analysis as printed by the pre-processing script (cf. Tables 3.1 and 4.3).

	Before scaling			After scaling		
Feature	Min	Max	Mean	Min	Max	Mean
`Init_Win_bytes_backward`	−1	65 535	2 493	0	1	0.0381
`Init_Win_bytes_forward`	−1	65 535	6 354	0	1	0.0970
`Destination Port`	0	65 529	8 152	0	1	0.1244
`Flow Duration`	−1	119 999 993	12 876 964	0	1	0.1073
`Bwd IAT Total`	0	120 000 000	11 635 099	0	1	0.0970
`PSH Flag Count`	0	1	0.2469	0	1	0.2469

Table 4.4: Six example features of the training set before and after min-max scaling.

```
======================================================================
Top features by chi2: binary
======================================================================
 rank                feature  score   p_value
    1 Init_Win_bytes_backward   4525         0
    2          PSH Flag Count   2929         0
    3  Init_Win_bytes_forward   1617         0
    4          ACK Flag Count  238.4 8.943e-54
    5       Destination Port  185.4 3.256e-42
    6          Bwd IAT Total  100.4 1.238e-23
    7          Down/Up Ratio  75.32 3.998e-18
    8            Fwd IAT Std  68.76 1.115e-16
    9          Fwd PSH Flags  68.03 1.612e-16
   10   Bwd Packet Length Min  56.82 4.768e-14
   11                Idle Max   49.8 1.699e-12
   12               Idle Mean  48.52 3.268e-12
   13                Idle Min  46.57 8.852e-12
   14         URG Flag Count  44.97 1.996e-11
   15     Packet Length Mean  41.21 1.365e-10
   16    min_seg_size_forward  40.12 2.385e-10
   17  Bwd Packet Length Mean  39.02 4.202e-10
   18             Bwd IAT Max  38.44 5.655e-10
   19           Fwd IAT Total  38.11 6.678e-10
   20           Flow Duration  37.96 7.216e-10
```

(a) Binary target

```
======================================================================
Top features by chi2: multiclass
======================================================================
 rank                feature  score   p_value
    1 Init_Win_bytes_backward   4597         0
    2          PSH Flag Count   2937         0
    3  Init_Win_bytes_forward   1621         0
    4          ACK Flag Count    246  4.76e-53
    5       Destination Port  185.4   6.1e-40
    6          Bwd IAT Total  100.4 1.249e-21
    7          Down/Up Ratio   77.2 1.225e-16
    8            Fwd IAT Std  70.42 3.468e-15
    9         URG Flag Count  69.62 5.135e-15
   10          Fwd PSH Flags  68.03 1.129e-14
   11   Bwd Packet Length Min  56.82 2.803e-12
   12                Idle Max   49.8 8.797e-11
   13               Idle Mean  48.52  1.65e-10
   14   Bwd Packet Length Std  47.63 2.559e-10
   15                Idle Min  46.57 4.296e-10
   16     Packet Length Mean  42.83 2.675e-09
   17  Bwd Packet Length Mean   41.5 5.128e-09
   18    min_seg_size_forward  40.12 1.004e-08
   19     Average Packet Size  39.13 1.627e-08
   20             Bwd IAT Max  38.67 2.044e-08
```

(b) Multiclass target

Figure 4.3: The 20 best features by χ^2 as printed by the pre-processing script (cf. Table 4.5).

4.2 Interpretation

Several of the top features are specific to the 2017 testbed. The initial TCP window sizes (`Init_Win_bytes_backward` and `_forward`) dominate, but they depend on the operating systems and TCP stacks of the hosts in the 2017 testbed, not on the attack. `Destination Port` is a shortcut: every

Rank	Binary feature	χ^2	Multiclass feature	χ^2
1	`Init_Win_bytes_backward`	4525.18	`Init_Win_bytes_backward`	4596.70
2	`PSH Flag Count`	2928.73	`PSH Flag Count`	2937.06
3	`Init_Win_bytes_forward`	1616.52	`Init_Win_bytes_forward`	1620.67
4	`ACK Flag Count`	238.36	`ACK Flag Count`	246.02
5	`Destination Port`	185.37	`Destination Port`	185.37
6	`Bwd IAT Total`	100.41	`Bwd IAT Total`	100.44
7	`Down/Up Ratio`	75.32	`Down/Up Ratio`	77.20
8	`Fwd IAT Std`	68.76	`Fwd IAT Std`	70.42
9	`Fwd PSH Flags`	68.03	`URG Flag Count`	69.62
10	`Bwd Packet Length Min`	56.82	`Fwd PSH Flags`	68.03
11	`Idle Max`	49.80	`Bwd Packet Length Min`	56.82
12	`Idle Mean`	48.52	`Idle Max`	49.80
13	`Idle Min`	46.57	`Idle Mean`	48.52
14	`URG Flag Count`	44.97	`Bwd Packet Length Std`	47.63
15	`Packet Length Mean`	41.21	`Idle Min`	46.57
16	`min_seg_size_forward`	40.12	`Packet Length Mean`	42.83
17	`Bwd Packet Length Mean`	39.02	`Bwd Packet Length Mean`	41.50
18	`Bwd IAT Max`	38.44	`min_seg_size_forward`	40.12
19	`Fwd IAT Total`	38.11	`Average Packet Size`	39.13
20	`Flow Duration`	37.96	`Bwd IAT Max`	38.67

Table 4.5: The 20 best features by χ^2 score (min-max-scaled training set), for the binary and the multiclass target.

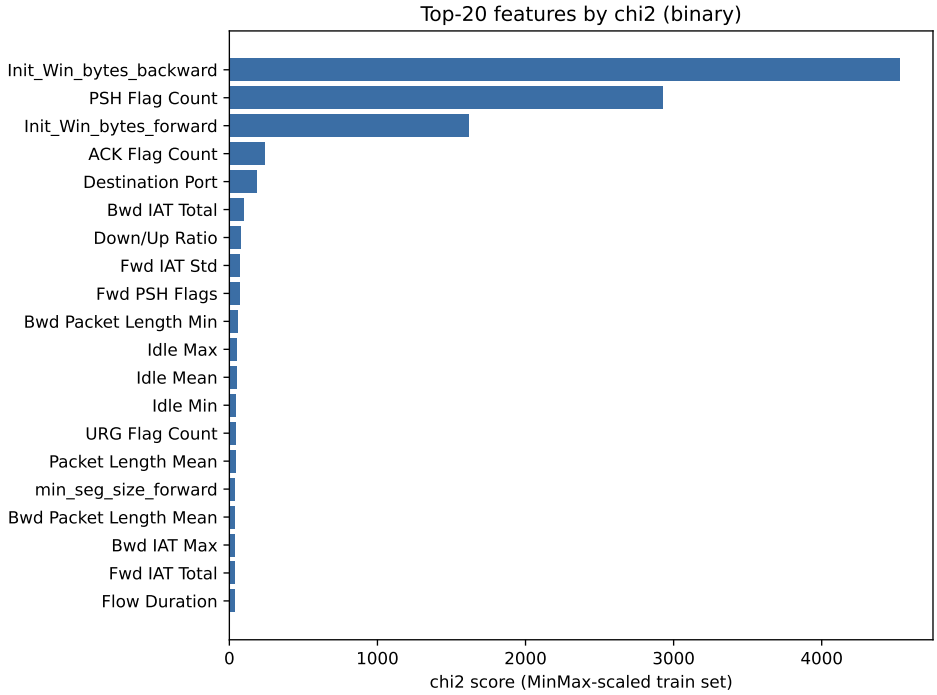


Figure 4.4: Top-20 flow features by χ^2 score for binary classification (BENIGN vs. ATTACK), computed on the min-max-scaled training set.

attack targets port 80, so a model can learn "port 80 is suspicious" instead of the attack behavior. The flag features (`PSH`, `ACK`, `URG`, `Fwd PSH Flags`) capture HTTP request and response patterns, and the timing features (`IAT`, `Idle`, `Flow Duration`) plausibly capture the regular pacing of the automated

attacks. We expect the testbed-specific features to generalize poorly to our own traffic. The χ^2 test on min-max-scaled continuous data is also only a heuristic: it treats scaled values like frequencies, and heavy-tailed features such as `Flow Bytes/s` are squeezed towards 0, which lowers their scores. The ranking therefore depends on the scaling.

5 Task 4: Machine Learning Classifiers

5.1 Setup

We train Logistic Regression (LR, `max_iter=5000`) and a Decision Tree (DT, `random_state=42`, unpruned) with scikit-learn [Ped+11]. Each is trained in two variants, default and `class_weight="balanced"`, and for two targets: binary (BENIGN vs. ATTACK) and multiclass (four labels). This gives eight models. Every model is one scikit-learn `Pipeline`: `MinMaxScaler(clip=True)`, `SelectKBest(chi2, k=20)` and the classifier (Listing 5.1). Scaler and selector are fitted on the training split only (114,925 flows). All models are evaluated on the held-out test split of 49,254 flows, of which 643 are attacks. The selected features of every saved pipeline equal the Task 3 lists exactly, and all four LR models converged well below the iteration limit. Inference time is the median of five timed `predict` calls on the whole test split after one warm-up call, scaled to 1,000 flows.

Listing 5.1: `build_pipeline()` and `make_classifier()` in `src/lab2ids/models.py`

```python
def build_pipeline(clf: Any, k: int = K) -> Pipeline:
    return Pipeline([
        ("scale", MinMaxScaler(clip=True)),
        ("select", SelectKBest(chi2, k=k)),
        ("clf", clf),
    ])
def make_classifier(model: str) -> Any:
    makers = {
        "lr_default": lambda: LogisticRegression(max_iter=LR_MAX_ITER),
        "lr_balanced": lambda: LogisticRegression(max_iter=LR_MAX_ITER, class_weight="balanced"),
        "dt_default": lambda: DecisionTreeClassifier(random_state=RANDOM_STATE),
        "dt_balanced": lambda: DecisionTreeClassifier(
            random_state=RANDOM_STATE, class_weight="balanced"),
    }
    return makers[model]()
```

5.2 Binary Results

The default Decision Tree is the best binary model (Table 5.1): it detects all 643 attacks (recall 1.0000) with 18 false positives and an attack F1 of 0.9862. Its balanced variant is almost identical. Logistic Regression does worse in both variants. The default LR misses 94 attacks (recall 0.8538). With `class_weight="balanced"`, LR reaches a recall of 0.9907, but its precision drops to 0.2349. It raises 2,075 false alarms, ten times as many as the default LR (201). Figure 5.2 (a) and (b) show the confusion matrices of the default DT and the balanced LR.

Model	Acc.	Macro-F1	Prec.	Rec.	F1	FP	FN	Fit (s)	Inf. (ms/1k)
LR default	0.9940	0.8926	0.7320	0.8538	0.7882	201	94	5.74	0.708
LR balanced	0.9577	0.6789	0.2349	0.9907	0.3797	2 075	6	23.26	0.672
DT default	**0.9996**	**0.9930**	**0.9728**	**1.0000**	**0.9862**	**18**	**0**	**0.36**	**0.667**
DT balanced	0.9996	0.9925	0.9815	0.9891	0.9853	12	7	0.35	0.683

Table 5.1: Binary results on the test split (Prec., Rec., F1 refer to the attack class; FP/FN are false positives and false negatives). Best model in bold.

```
======================================================================
binary_dt_default
======================================================================
       class  precision  recall     f1  support
      BENIGN     1.0000  0.9996 0.9998    48611
      ATTACK     0.9728  1.0000 0.9862      643
   macro avg     0.9864  0.9998 0.9930    49254
weighted avg        NaN     NaN 0.9996    49254
    accuracy        NaN     NaN 0.9996    49254
accuracy = 0.9996 | macro-F1 = 0.9930 | weighted-F1 = 0.9996
Confusion matrix (rows = true, cols = predicted):
             pred BENIGN  pred ATTACK
true BENIGN        48593           18
true ATTACK            0          643
fit = 0.36 s | inference = 0.667 ms per 1,000 flows

----------------------------------------------------------------------
```

Figure 5.1: Console output of the training script for the chosen model, the default Decision Tree (binary).

5.3 Multiclass Results

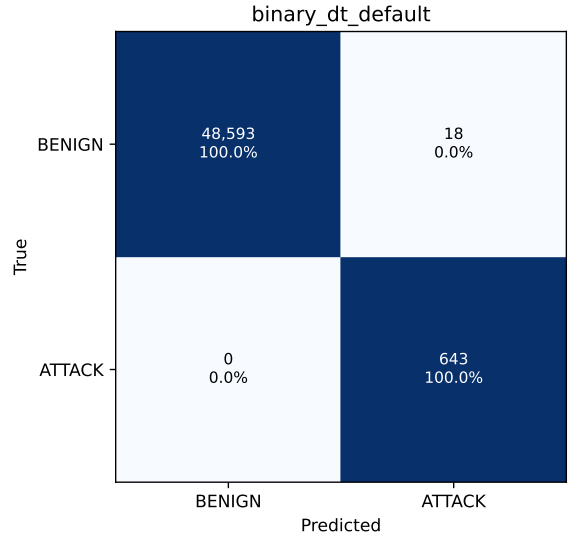
In the multiclass setting, the default Decision Tree again leads, with a macro-F1 of 0.7290 (Table 5.2). The default Logistic Regression reaches an accuracy of 98.54 % but detects *no* attack at all (F1 of 0.0000 for all three attack classes, macro-F1 0.2482): it predicts almost everything as BENIGN (Figure 5.2 (d)). This is the accuracy paradox, since 98.7 % of the data is benign anyway. Class weighting raises the LR macro-F1 only to 0.3191 and lowers the DT macro-F1 to 0.5642. Within the DT, Brute Force and XSS are frequently confused (Figure 5.2 (c)): 103 Brute Force flows are predicted as XSS and 97 XSS flows as Brute Force. Both attacks are automated HTTP requests from the same client to the same web server, and flow statistics cannot see the payload, which is the only thing that really differs. SQL injection reaches an F1 of 0.9091, but with only 6 test flows (5 detected) this number is statistically weak: one flow more or less changes it by about 0.1.

Model	Acc.	Macro-F1	BENIGN	Brute Force	XSS	SQLi	Fit (s)	Inf. (ms/1k)
LR default	0.9854	0.2482	0.9926	0.0000	0.0000	0.0000	11.67	0.688
LR balanced	0.9321	0.3191	0.9689	0.0781	0.2233	0.0061	26.40	0.835
DT default	0.9923	0.7290	0.9982	0.6126	0.3960	0.9091	0.73	0.701
DT balanced	0.9913	0.5642	0.9977	0.5827	0.3762	0.3000	0.77	0.675

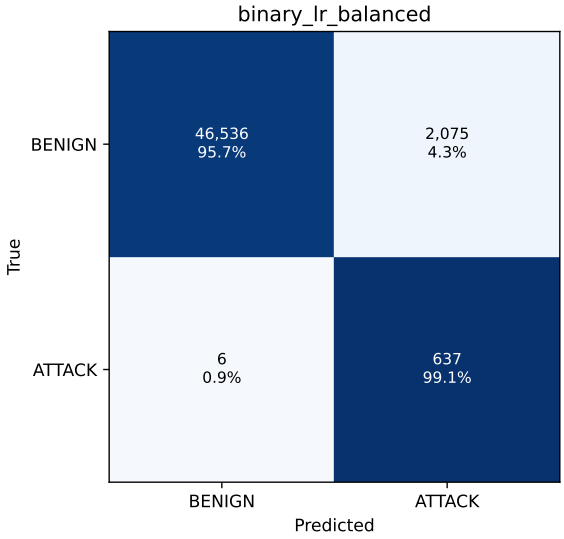
Table 5.2: Multiclass results on the test split. The columns BENIGN to SQLi are per-class F1 scores.

5.4 Interpretation

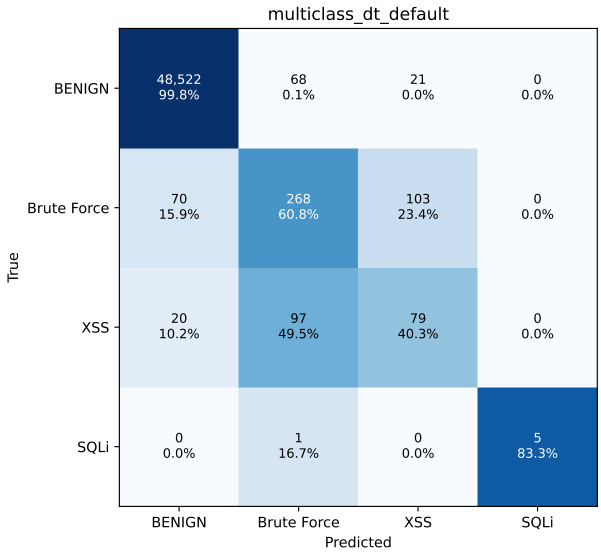
The Decision Tree beats Logistic Regression on both targets: attack F1 of 0.986 against at best 0.788, and multiclass macro-F1 of 0.729 against 0.319. Attacks occupy narrow value ranges of a few features (small TCP windows, port 80, flag counts), and a tree isolates such ranges with axis-aligned thresholds. LR draws a single linear boundary through heavy-tailed, min-max-compressed features and cannot. Class weighting buys LR recall at the price of precision; for the DT it hardly changes the binary result and hurts the multiclass result. The binary Decision Tree (default) is the model we use in the real-time part.



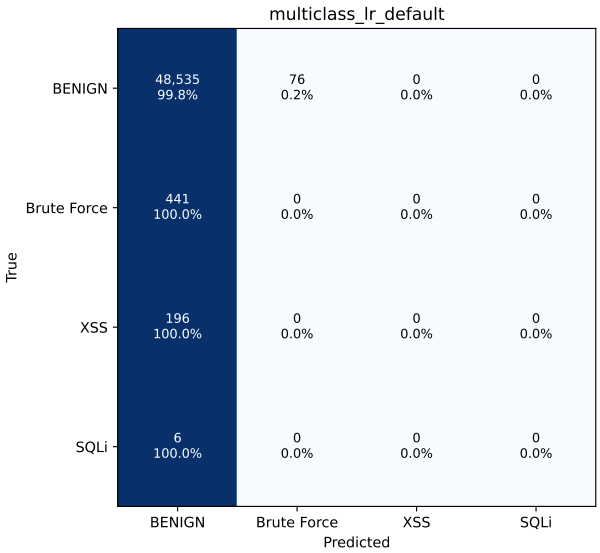
(a) Decision Tree (binary): all 643 attacks detected, 18 false positives.



(b) Balanced Logistic Regression: 99 % attack recall at the cost of 2,075 false alarms.



(c) Decision Tree (4 classes): Brute Force and XSS are frequently confused.



(d) Logistic Regression (4 classes) predicts almost everything as BENIGN: 98.5 % accuracy, 0 attacks detected.

Figure 5.2: Confusion matrices of four of the eight models (test split).

6 Task 4: Real-Time Classification

6.1 Setup

For the real-time part, we use the binary default Decision Tree. It has the best attack F1, missed no attack on the test split, and has a false-positive rate of $18/48{,}611 = 0.037\,\%$. The detector loads the saved pipeline unchanged, so training and serving use exactly the same scaling, selection and classifier. Figure 6.1 shows the data flow. The input is the flow CSV of our own capture (Chapter 2): 569 flows produced by CICFlowMeter V4, whereas the training data comes from an older version. The detector streams the flows one by one in timestamp order, maps the V4 column names to the training names

(all 77 features map), drops the `Protocol = 0` rows, replaces infinite and NaN values by 0 and classifies each flow (Listing 6.1). The flow CSV is a command-line argument, so the detector can be pointed at any other V4 flow file.

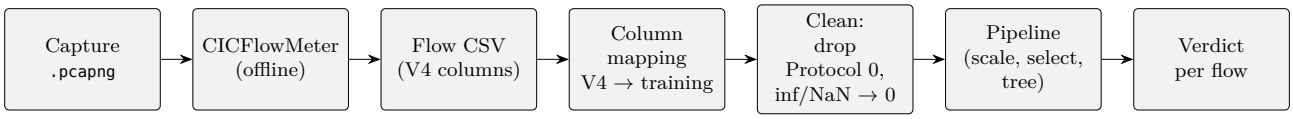


Figure 6.1: Real-time architecture. Flows are streamed row by row in timestamp order.

Listing 6.1: Excerpt of `Detector.process()` in `src/lab2ids/realtime/detector.py`: column mapping, Protocol-0 drop, feature check, inf/NaN replacement and prediction (some lines omitted)

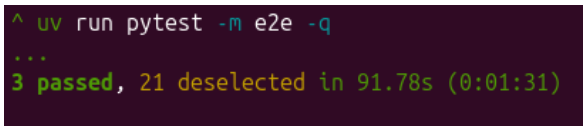
```
        df = map_v4_to_iscx(df_v4)
        # 2. drop Protocol-0 (non TCP/UDP / unparsed) rows
        n_proto0 = 0
        if "Protocol" in df.columns:
            keep = pd.to_numeric(df["Protocol"], errors="coerce").fillna(0) != 0
            n_proto0 = int((~keep).sum())
            df = df[keep]
        missing = [c for c in self.input_features if c not in df.columns]
        if missing:
            raise ValueError(f"missing required feature columns: {missing}")
        x = df[self.input_features].apply(pd.to_numeric, errors="coerce").astype(float)
        # 5. inf/NaN -> 0
        bad = ~np.isfinite(x.to_numpy())
        n_replaced = int(bad.sum())
        x = x.mask(bad, 0.0)
        if len(x):
            pred = self.pipeline.predict(x)
            proba = self.pipeline.predict_proba(x)
            is_alert = np.array([lab not in BENIGN_NAMES for lab in labels], dtype=bool)
```

6.2 Correctness: Parity Test

To prove that the real-time path does not change the model's behavior, we converted the 49,254-flow test split to the V4 column names and streamed it through the detector. The predictions are identical, row for row, to those of the offline pipeline, both for batches of 1,000 flows and for batches of one flow. The attack F1 recomputed from the streamed predictions equals the offline value of Table 5.1. Figure 6.2 shows the test evidence: 21 unit tests and 3 full-dataset tests (Task 3 numbers, Task 4 regression guards and this parity test) pass.



(a) Unit tests: 21 passed.



(b) Full-dataset tests: 3 passed.

Figure 6.2: Test evidence for Tasks 3 and 4.

6.3 Results on Our Capture

Of 569 flows, 22 were dropped (protocol 0) and 547 were classified: 544 as BENIGN and 3 as ATTACK, an alert rate of 0.55 % (Table 6.1, Figure 6.3). Our traffic is benign by construction, so all three alerts are false positives. For the same reason, we can measure the false-positive rate on our network but not the detection rate.

Quantity	Value
Flows read	569
Protocol-0 flows dropped	22
inf/NaN cells replaced	0
Flows classified	547
Classified BENIGN	544
ATTACK alerts	3
Alert rate	0.55 %
Latency per flow, mean (ms)	14.203
Latency per flow, median (ms)	13.510

Table 6.1: Real-time run on `task2_lab2.pcap_Flow.csv`.

```
04/11/2026 04:10:43 PM  10.241.62.200:53370 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:43 PM  10.241.62.200:52751 → 155.133.226.7:443/6  BENIGN  p=0.00
04/11/2026 04:10:43 PM  10.241.62.200:53493 → 155.133.226.39:443/6  BENIGN  p=0.00
04/11/2026 04:10:43 PM  10.241.62.200:50018 → 52.85.65.82:443/6  BENIGN  p=0.00
04/11/2026 04:10:43 PM  10.241.62.200:58295 → 155.133.226.4:443/6  BENIGN  p=0.00
04/11/2026 04:10:43 PM  10.241.62.200:34550 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:44 PM  10.241.62.200:42821 → 162.254.197.28:443/6  BENIGN  p=0.00
04/11/2026 04:10:46 PM  10.241.62.200:60211 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:46 PM  10.241.62.200:40970 → 84.53.190.129:443/6  BENIGN  p=0.00
04/11/2026 04:10:47 PM  10.241.62.200:42421 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:47 PM  10.241.62.200:54040 → 199.232.191.52:443/6  BENIGN  p=0.00
04/11/2026 04:10:47 PM  10.241.62.200:54641 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:47 PM  10.241.62.200:40970 → 84.53.190.129:443/6  BENIGN  p=0.00
04/11/2026 04:10:47 PM  10.241.62.200:40970 → 84.53.190.129:443/6  BENIGN  p=0.00
04/11/2026 04:10:47 PM  10.241.62.200:40970 → 84.53.190.129:443/6  BENIGN  p=0.00
04/11/2026 04:10:47 PM  10.241.62.200:51538 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:48 PM  10.241.62.200:54054 → 199.232.191.52:443/6  BENIGN  p=0.00
04/11/2026 04:10:48 PM  10.241.62.200:37559 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:48 PM  10.241.62.200:48389 → 10.241.62.246:53/17  BENIGN  p=0.00
04/11/2026 04:10:48 PM  10.241.62.200:57413 → 199.232.191.52:443/6  BENIGN  p=0.00

======================================================================
Summary
======================================================================
model                    : binary_dt_default
flows read               : 569
Protocol-0 rows dropped  : 22
inf/NaN cells replaced   : 0
flows classified         : 547
BENIGN                   : 544
ATTACK (alerts)          : 3
alert rate               : 0.55 %
latency per flow (ms)    : mean 14.203 | median 13.510
Alert flows:
  04/11/2026 04:10:40 PM  10.241.62.200:54151 → 92.226.1.27:80/6  ATTACK  p=1.00
  04/11/2026 04:10:40 PM  10.241.62.200:55257 → 92.226.1.27:80/6  ATTACK  p=1.00
  04/11/2026 04:10:40 PM  10.241.62.200:51729 → 92.226.1.27:80/6  ATTACK  p=1.00
```

Figure 6.3: Output of the real-time detector: streamed flow verdicts, the summary (569 read, 22 dropped, 547 classified, 3 alerts = 0.55 %) and the three alerts in red.

6.4 The False Positives

We checked the three alerts against the capture. At 16:10:40, the Steam client sent seven parallel requests `GET /appinfo/<id>/sha/<hash>.txt.gz` over plain HTTP to `clientconfig.akamai.steamstatic.com` (`92.226.1.27:80`, Figure 6.4). All received `200 OK` with small gzip files (0.8–3.7 kB). The seven flows are nearly identical: server initial window `Init_Win_bytes_backward` = 507, client window 64,240, port 80, about 101 ms duration and 2 PSH flags.

The tree flags them because its first splits route all seven into the region "small server initial window ($235 < 507 \leq 1{,}146$) and destination port ≤ 85". In the training data, this region holds attacks: the Brute Force and XSS servers had small windows (235, 236 and 1,150), and every attack targets port 80. These are the shortcut features of Section 4.2. Only 3 of the 7 flows are flagged because the

final verdict comes from a split at depth 15 on packet-length features, where the unpruned tree lets 4 near-identical flows pass and flags 3. The tree has overfitted to these fine splits.

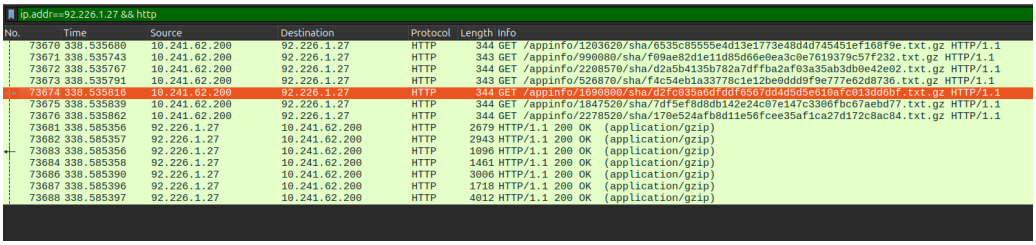


Figure 6.4: The seven parallel Steam requests in Wireshark (display filter at the top): `GET /appinfo/…` to the Steam CDN at 92.226.1.27, all answered with `200 OK`. The host name, `clientconfig.akamai.steamstatic.com`, comes from the HTTP Host header.

6.5 Latency and Scope

Classifying one flow at a time takes 14.2 ms on average (median 13.5 ms) in our recorded run; an earlier run measured 11.6 ms, so there is run-to-run variance. This is per-call pandas and scikit-learn overhead: batched, the same model needs 0.67 ms per 1,000 flows. In practice, the detection delay is dominated by CICFlowMeter, which emits a flow only when it ends (FIN or RST) or times out (5 s activity, 120 s flow timeout). "Real time" here means streaming the flows of a recorded capture through the detector one by one; we do not capture packets live.

7 Discussion and Conclusion

Shortcut features and domain shift. The Decision Tree separates attacks from benign flows almost perfectly on the CIC-IDS2017 test split (attack F1 0.986, false-positive rate 0.037 %). On our own benign traffic, however, the false-positive rate is 0.55 %, about 15 times higher. The false alarms come from the shortcut features of Section 4.2, `Destination Port` and `Init_Win_bytes_*`, applied to automated plain-HTTP bursts that resemble the Selenium-driven attacks. The model partly learned "automated HTTP to port 80 from a small-window server", not the attack itself. Other differences add to the shift: the training flows come from an older CICFlowMeter version and a 2017 testbed with different hosts and a different application mix, whereas our capture contains QUIC and Steam traffic. The data itself also has quality problems, such as the broken label encoding, the duplicated column, infinite values, about 6,000 duplicate rows and two TCP flag columns that are identical. They match issues reported in the literature [ERJ21], and each of them needed its own cleaning step in Chapter 4.

Accuracy paradox. Multiclass Logistic Regression reaches 98.54 % accuracy while detecting no attack, because 98.7 % of the flows are benign. Class weighting does not solve the imbalance either. It raises LR's attack recall from 0.85 to 0.99 but multiplies the false alarms by ten, and it lowers the multiclass macro-F1 of the tree.

Limitations. We use a single train/test split without cross-validation. The tree is unpruned, and its deep, fine splits show up in the false-positive analysis. SQL injection has only 6 test flows, so its metrics are statistically weak. Brute Force and XSS cannot be separated well from flow statistics alone. Our own capture contains no attacks, so only the false-positive rate can be measured on our network. Dropping rows with infinite values slightly changes the benign distribution (0.08 %), and min-max scaling is sensitive to outliers. Finally, the real-time part replays recorded flows and is not live capture.

Possible improvements (not implemented). Pruning the tree (`max_depth`, `min_samples_leaf`), dropping the testbed-specific features (`Destination Port`, `Init_Win_bytes_*`), and adding benign traffic from the deployment network to the training data should reduce the false positives.

Conclusion. We turned the Thursday web-attack file of CIC-IDS2017 into a clean set of 164,179 flows with 60 features, selected 20 features per target, and trained Logistic Regression and Decision Tree classifiers. The Decision Tree outperforms Logistic Regression on both targets and detects all attacks in the binary test split, while accuracy hides that default multiclass LR detects none. On our own benign traffic, the tree's false-positive rate rose from 0.037 % to 0.55 %, so its benchmark score does not carry over to our network.

Bibliography

[ERJ21] Gints Engelen, Vera Rimmer, and Wouter Joosen. "Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study." In: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009 (cit. on pp. 6, 14).

[CIC] *IDS 2017 Dataset*. Canadian Institute for Cybersecurity, University of New Brunswick. URL: https://www.unb.ca/cic/datasets/ids-2017.html (visited on 11/04/2026) (cit. on p. 4).

[CFM] Arash Habibi Lashkari. *CICFlowMeter*. URL: https://github.com/ahlashkari/CICFlowMeter (visited on 11/04/2026) (cit. on p. 1).

[Las+17] Arash Habibi Lashkari et al. "Characterization of Tor Traffic Using Time Based Features." In: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*. 2017, pp. 253–262. DOI: 10.5220/0006105602530262 (cit. on pp. 1, 4).

[Ped+11] Fabian Pedregosa et al. "Scikit-learn: Machine Learning in Python." In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830. URL: https://jmlr.org/papers/v12/pedregosa11a.html (cit. on pp. 4, 9).

[SLG18] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, pp. 108–116. DOI: 10.5220/0006639801080116 (cit. on p. 4).

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese MasterLab selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe und alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, als solche gekennzeichnet sind, sowie, dass ich die MasterLab in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Passau, 6. November 2026

Justlyn Kaniyanthara Johnson

Aayush Kapoor